



FCT

FACULDADE DE CIÊNCIAS
E TECNOLOGIA
UNIVERSIDADE DOS AÇORES

**FACULDADE DE CIÊNCIAS E TECNOLOGIA DA UNIVERSIDADE DOS AÇORES -
CAMPUS de Ponta Delgada
COORDENAÇÃO DE CURSO DE PÓS-GRADUAÇÃO EM PROGRAMAÇÃO,
ROBÓTICA E INTELIGÊNCIA ARTIFICIAL
Laboratório de Aplicações em Robótica e Aprendizagem**

OpenBalance

Plataforma de Equilíbrio com Visão Computacional e Controlo PID

João Pavão

**FCT - UAC
2025**

RESUMO

Este relatório apresenta o desenvolvimento do projeto OpenBalance, uma plataforma robótica experimental concebida para equilibrar uma bola em tempo real, controlando dinamicamente a inclinação da sua superfície. O sistema recorre a visão computacional (OpenCV) para detetar a posição da bola e aplica algoritmos de controlo PID para mantê-la centrada. Desenvolvido no contexto da unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA), o projeto integra conhecimentos de mecânica, eletrónica, automação e inteligência artificial aplicada. A estrutura física é composta por componentes impressos em 3D, articulações mecânicas e suportes que garantem graus de liberdade bem definidos. No domínio da eletrónica, são utilizados servomotores de alto torque, um driver PCA9685, um Arduino UNO e uma fonte de alimentação estabilizada. A deteção visual e o controlo são integrados numa interface gráfica interativa em Python, desenvolvida com CustomTkinter. Os testes realizados demonstraram estabilidade, resposta eficiente ao controlo e elevado potencial educativo, validando a proposta como recurso didático modular e reproduzível.

Palavras-chave: Visão Computacional, Controlo PID, Arduino, Mecatrónica, Interface Gráfica, Plataforma de Equilíbrio, Python

ABSTRACT

This report presents the development of OpenBalance, an experimental robotic platform designed to balance a ball in real time by dynamically controlling the inclination of its surface. The system uses computer vision (OpenCV) to detect the ball's position and applies PID control algorithms to recenter it continuously. Developed within the scope of the Laboratory of Robotics and Learning Applications (PRIA) course, the project integrates knowledge from mechanics, electronics, automation, and applied artificial intelligence. The physical structure consists of 3D-printed components, mechanical joints, and supports that define specific degrees of freedom. On the electronics side, the system uses high-torque servomotors, a PCA9685 servo driver, an Arduino UNO, and a regulated power supply. Visual detection and control are managed through an interactive graphical interface in Python, built with CustomTkinter. The conducted tests demonstrated system stability, effective response to control inputs, and strong educational potential, validating the platform as a modular and reproducible teaching tool.

Keywords: Computer Vision, PID Control, Arduino, Mechatronics, Graphical Interface, Ball-Balancing Platform, Python

1 INTRODUÇÃO

O presente relatório documenta o desenvolvimento do projeto OpenBalance, uma plataforma robótica experimental concebida no âmbito da unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA) da Universidade dos Açores. O sistema tem como principal objetivo demonstrar, de forma visual e interativa, os fundamentos do controlo automático aplicados a um problema clássico: o equilíbrio dinâmico de uma bola sobre uma superfície inclinável.

A ideia central consiste em permitir que uma bola se mantenha centrada numa plataforma bidimensional, cuja inclinação é controlada por dois servomotores de alto torque (modelo DM996), um em cada eixo (X e Y), acionados por um microcontrolador Arduino Uno através de um driver de 16 canais PCA9685. A posição da bola é monitorizada em tempo real através de uma webcam USB, com processamento de imagem realizado em Python utilizando a biblioteca OpenCV. A decisão sobre os ângulos de inclinação é determinada por algoritmos de controlo PID (Proporcional–Integral–Derivativo), com parâmetros ajustáveis por uma interface gráfica interativa desenvolvida com CustomTkinter. O sistema inclui ainda uma estrutura física desenhada em software CAD e fabricada com peças em PLA (impressão 3D) e contraplacado.

O OpenBalance não é apenas uma implementação técnica; constitui também uma ferramenta pedagógica e de código aberto, pensada para facilitar o ensino e a aprendizagem de conceitos fundamentais de automação, controlo, robótica e visão computacional. A modularidade do sistema permite que seja facilmente replicado, adaptado e expandido, por exemplo para sistemas de maior complexidade como plataformas de Stewart (6 graus de liberdade), ou mesmo para investigações em controlo adaptativo.

Neste contexto, considera-se também, de forma exploratória, a viabilidade futura da substituição ou extensão do controlo PID por abordagens baseadas em aprendizagem por reforço (Reinforcement Learning – RL), em que um agente aprende a atuar com base na interação com o ambiente. Esta perspetiva abre caminho para o desenvolvimento de controladores autónomos capazes de se adaptarem a diferentes dinâmicas do sistema sem exigir modelos matemáticos explícitos, alargando assim o potencial do OpenBalance como plataforma de investigação e experimentação em inteligência artificial aplicada à robótica.

Ao longo deste relatório serão apresentados os objetivos do projeto, os componentes utilizados, a estrutura de hardware e software desenvolvida, bem como as estratégias de calibração e os resultados obtidos. Será ainda feita uma reflexão sobre as potencialidades educativas do projeto e as perspetivas futuras para a sua evolução.

2 OBJETIVO

O projeto OpenBalance tem como objetivo principal o desenvolvimento de uma plataforma robótica educativa que demonstre o funcionamento de sistemas de controlo em tempo real, com ênfase no controlo PID e na visão computacional. Pretende-se criar um modelo físico interativo para o estudo de sistemas dinâmicos, promovendo a aplicação prática de conceitos abordados na unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA).

Objetivos específicos:

- Construir uma plataforma mecânica funcional com dois eixos de inclinação (X e Y) utilizando servomotores de precisão;
- Integrar um sistema de aquisição de imagem em tempo real com webcam USB e biblioteca OpenCV para deteção da posição da bola;
- Implementar algoritmos de controlo PID para estabilizar a bola no centro da plataforma;
- Desenvolver uma interface gráfica interativa em Python (CustomTkinter) para controlo de parâmetros PID, deteção de cor (HSV) e visualização em tempo real;
- Testar e calibrar o sistema, otimizando o desempenho físico e algorítmico;
- Documentar todo o processo de desenvolvimento e publicar o projeto num repositório open source no GitHub;
- Explorar possibilidades de expansão futura, incluindo:
 - Extensão para plataformas com três ou mais atuadores;
 - Conceção de um sistema tipo Stewart com seis graus de liberdade;
 - Implementação de comportamentos proativos baseados em visão;
 - Integração com algoritmos de aprendizagem por reforço;
 - Substituição da visão por sensores resistivos ou táteis integrados na plataforma.

3 CONTEXTO

O projeto OpenBalance insere-se no âmbito da unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA), do curso de pós-graduação em Programação, Robótica e Inteligência Artificial da Universidade dos Açores. Trata-se de uma plataforma robótica experimental cujo objetivo é demonstrar, em tempo real, o funcionamento de sistemas de controlo aplicados ao equilíbrio dinâmico de uma bola sobre uma superfície inclinável.

A estrutura base do sistema é composta por uma plataforma física bidimensional, fabricada com componentes em PLA (impresso em 3D) e contraplacado leve, sustentada por dois servomotores de alto torque (modelo DM996). Estes são responsáveis por controlar a inclinação nos eixos X e Y. O controlo dos servomotores é realizado por um microcontrolador Arduino UNO, que comunica com um módulo PCA9685 para gerar os sinais PWM adequados.

A deteção da posição da bola é feita através de uma webcam USB montada verticalmente sobre a plataforma. As imagens captadas são processadas em tempo real com Python, utilizando a biblioteca OpenCV para segmentação por cor e localização do centro da bola. Esta informação é

utilizada como entrada para algoritmos PID (Proporcional–Integral–Derivativo), que calculam as correções necessárias para recentrar a bola na plataforma. Os parâmetros do PID são atualmente definidos diretamente no código Arduino, antes da execução.

O sistema é gerido a partir de uma interface gráfica desenvolvida com CustomTkinter, que permite ao utilizador ajustar os parâmetros de deteção HSV, observar o vídeo processado em tempo real e controlar o estado dos motores. Embora a interface já contenha espaço reservado para futuras opções de calibração e controlo PID direto, estas funcionalidades ainda não estão ativas na versão atual. A comunicação entre o computador e o Arduino é realizada via porta serial USB, garantindo uma separação clara entre o processamento lógico e o controlo físico.

O projeto foi inteiramente desenvolvido com ferramentas open source, sendo reproduzível em qualquer ambiente compatível com Python, Arduino IDE e uma webcam convencional. O desenvolvimento em Python foi realizado com o editor Thonny, por se tratar de uma IDE leve, intuitiva e especialmente eficaz para projetos educativos ou compactos. Já o código dos servomotores e controlo PID foi implementado na Arduino IDE, assegurando compatibilidade e facilidade de integração com o hardware utilizado. Este contexto favorece a experimentação prática de conceitos de robótica, visão computacional e controlo automático, numa plataforma didática, acessível e extensível.

4 DESENVOLVIMENTO E RESULTADOS

O desenvolvimento do projeto OpenBalance envolveu várias fases complementares, desde a construção da estrutura física até à implementação dos sistemas de controlo e interface. A plataforma foi concebida com base em princípios de modularidade, acessibilidade e reprodutibilidade, utilizando componentes simples e ferramentas open source.

Estrutura física e montagem

A estrutura da plataforma foi construída com contraplacado leve como base móvel e componentes impressos em PLA para os suportes e braços articulados. Dois servomotores DM996 (15 kg/cm) foram montados em estruturas laterais verticais, permitindo a inclinação controlada da superfície nos eixos X e Y. A plataforma repousa sobre os braços por meio de um pivô central côncavo, proporcionando liberdade de movimento com estabilidade. Foram integrados encaixes tipo cavilha nos braços impressos para evitar deslizamento lateral e garantir alinhamento mecânico.

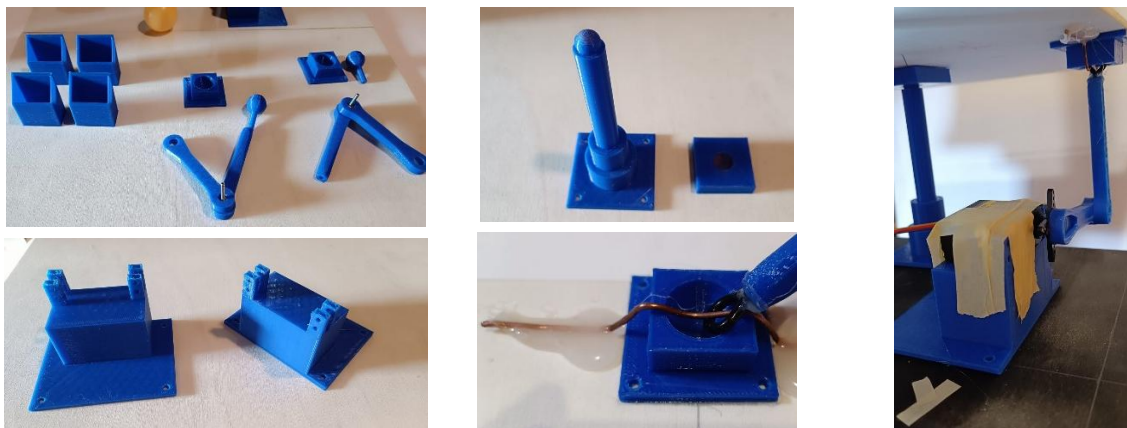


Figura 1 – Vários componentes que formam a estrutura da plataforma

A superfície visível da plataforma foi pintada de preto com uma cruz central (destinada ao alinhamento da webcam e a plataforma) e uma borda de madeira natural (~4 mm), otimizando o contraste visual para a detecção por visão computacional.

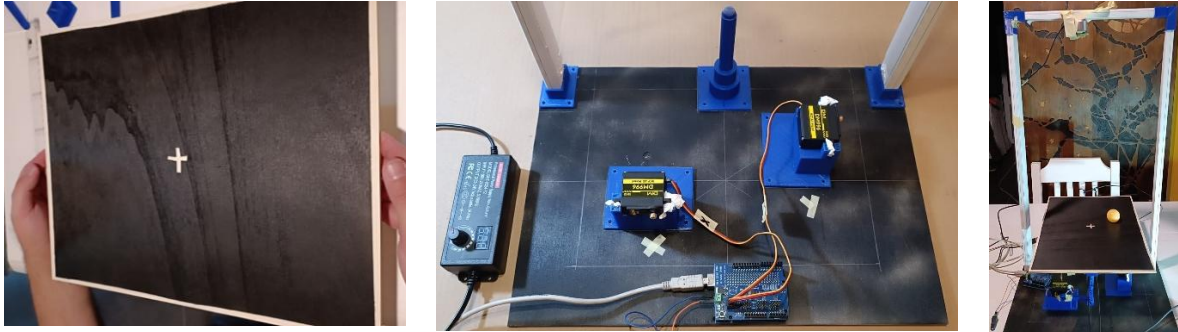
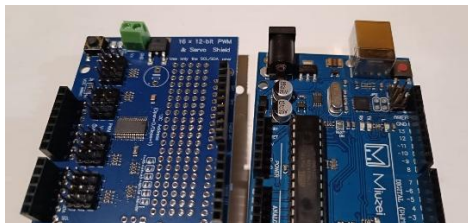


Figura 2 – Montagem da estrutura – etapas finais

Sistema eletrônico

O controlo dos servomotores foi realizado por um Arduino UNO, ligado a um driver PCA9685 de 16 canais (via I²C), que gera os sinais PWM com precisão. A alimentação dos motores é feita através de uma fonte regulável de 6 V / 72 W, garantindo potência estável sem sobrecarregar o Arduino. A comunicação com o computador é feita via porta serial USB. Para a deteção da bola, foi utilizada uma webcam USB genérica, montada numa calha técnica posicionada sobre a plataforma, garantindo uma visão perpendicular e estável.



Driver PCA9685 e Arduino Uno



Webcam USB



Fonte regulável

Figura 3 – Vários componentes da parte eletrónica

Calibração dos servomotores

Para assegurar que os movimentos da plataforma respeitavam os limites mecânicos e funcionavam de forma previsível, foi realizado um processo de calibração manual dos servos. Inicialmente, cada servo foi posicionado a 90° (posição neutra), e a montagem foi alinhada para que esta posição correspondesse a uma superfície nivelada. Através de um sketch dedicado de calibração, os ângulos máximos e mínimos seguros foram determinados por teste manual:

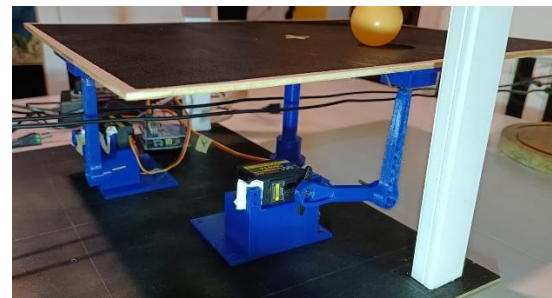
- Eixo X: 80° a 120°
- Eixo Y: 70° a 110°

Estes valores foram então codificados no firmware Arduino, garantindo que os comandos de controlo respeitam sempre os limites físicos da estrutura.

```

=== Calibração de Servos via PCA9685 ===
Comandos: X<ang> ou Y<ang> onde <ang> de 0 a 180
Exemplos: X0   X90   X180  Y45
-----
Servo X -> 95 graus
Servo X -> 85 graus
Servo X -> 88 graus
Servo Y -> 80 graus
Servo Y -> 85 graus
Servo X -> 89 graus
Servo X -> 90 graus
Servo X -> 92 graus
Servo Y -> 83 graus

```



Calibração dos motores servo feita no monitor serial do Arduino IDE

Calibração dos servo motores de modo a permitir a plataforma nivelada

Figura 4 – Calibração manual dos servomotores

Sintonização PID com o Método de Ziegler–Nichols

O controlo PID exige uma afinação cuidadosa dos seus três ganhos principais - proporcional (Kp), integral (Ki) e derivativo (Kd) - para garantir desempenho adequado. Esta afinação pode ser feita manualmente, de forma heurística, ou através de métodos sistemáticos. Um dos métodos mais reconhecidos e utilizados é o **método de Ziegler–Nichols (ZN)**, que oferece uma abordagem empírica eficaz mesmo na ausência de um modelo matemático completo do sistema.

Descrição do método de Ziegler–Nichols

O processo é constituído pelas seguintes etapas:

1. Desativar as componentes integral (Ki) e derivativa (Kd);
2. Aumentar o ganho proporcional (Kp) até o sistema oscilar de forma sustentada → obter o ganho último (Ku);
3. Medir o período da oscilação sustentada → Tu;
4. Aplicar as fórmulas empíricas de sintonização.

Tabela 1 – Fórmulas empíricas de sintonização segundo Ziegler–Nichols

Tipo de controlador	Kp	Ki	Kd
P	$0.50 \times Ku$	—	—
PI	$0.45 \times Ku$	$1.2 \times Kp / Tu$	—
PID	$0.60 \times Ku$	$2 \times Kp / Tu$	$Kp \times Tu / 8$

Resultados de testes com ganhos PID no eixo X

Durante a fase de afinação manual do PID, foram realizados testes variando o valor de Kp no eixo X, mantendo os ganhos Ki e Kd a zero e o eixo Y desativado. A Tabela 2 apresenta um resumo das observações obtidas.

Tabela 2 – Testes experimentais de controlo proporcional no eixo X

TESTES ao EIXO X										
Teste #	Eixo	Kp	Ki	Kd	Resposta Inicial	Oscilações	Erro Final	Tempo Estabilização	Estável?	Observações
1	X	0.2	0	0	suave	poucas	60 px (-)	pouco		
	X	0.3	0	0	suave	muitas		muito	bola entra em loop	
	X	0.5	0	0	rápida	muitas		muito muito		vai de lado a lado da plataforma
	X	0.4	0	0	media	muitas		muito tempo		toca menos vezes nos lados
	X	0.25	0	0	suave	poucas	113 px (X)	pouco	para ao fim de 10 s	mas toca algumas vezes
	X	0.28	0	0	suave					

Estes testes permitiram compreender a sensibilidade do sistema ao ganho proporcional e serviram de base para futuras afinações mais refinadas com os termos derivativo e integral ativados. Foram também realizados testes focados no eixo Y, mantendo o outro eixo inativo. Apenas após a afinação individual foi ativado o controlo PID nos dois eixos simultaneamente. A introdução dos ganhos integral (Ki) e derivativo (Kd) foi feita de forma progressiva e com base em tentativa e erro, a partir dos resultados observados com Kp isolado.

Os valores finais atualmente presentes no firmware do Arduino estão indicados na figura 5. Estes valores servem como referência inicial para afinação fina e podem ser ajustados em função da resposta observada.

```
// ===== Parâmetros PID por eixo, obtidos pelo método ZN =====
// -----EIXO X-----
const float Kp_X = 0.18, Ki_X = 0.015, Kd_X = 0.07;
// Sugestão: aumenta Kp_X para resposta mais rápida.
// Se houver oscilações, aumenta Kd_X ou reduz Kp_X.

// -----EIXO Y-----
const float Kp_Y = 0.14, Ki_Y = 0.015, Kd_Y = 0.05;
// Sugestão: se o eixo Y parecer mais lento,
// testa aumentar Ki_Y para 0.02 ou subir Kp_Y ligeiramente.
// =====
```

Figura 5 - Parâmetros PID definidos no firmware Arduino com base no método ZN

Está previsto para uma futura atualização da dashboard uma versão com funcionalidade ZN, que permita determinar rapidamente valores aproximados para os ganhos PID com base no comportamento oscilatório do sistema. Esta funcionalidade pretende facilitar a afinação prática, reduzindo o tempo de experimentação necessário.

Interface gráfica e interatividade

A interface gráfica foi desenvolvida com recurso à biblioteca CustomTkinter, utilizando uma arquitetura modular e moderna com tema escuro (ver figura 1). A aplicação apresenta três áreas principais:

- **Painel HSV:** sliders interativos para ajuste em tempo real da segmentação por cor (Hue, Saturation, Value), com possibilidade de aplicar presets pré-definidos e visualizar a máscara de deteção;
- **Área de vídeo:** visualização da imagem processada com sobreposição da bola detetada, do erro atual (X/Y), e de marcadores de referência (centro fixo e centro clicado com o rato);
- **Controlo do sistema:** botões para ligar e desligar motores, ativar/desativar o seguimento automático, redefinir o centro manualmente e seleccionar a porta COM do Arduino.



Painel HSV



Área de Visualização



Controlo do Sistema

Figura 6 – Vários componentes da parte eletrónica

A versão 1.0 da **dashboard OpenBalance** inclui detalhes como:

- Uma janela popup com dois gráficos combinados: **Erro X/Y vs Tempo** e **Trajetória da Bola (Y vs X)** em tempo real;
- Resolução de vídeo definida em **640×480** por defeito, assegurando boa qualidade visual com desempenho aceitável em máquinas atuais;
- Uma nova barra de estado com mensagens informativas sobre ações realizadas e erros detetados;
- Um botão "Reset Centro", que redefine o ponto de referência onde a bola deve convergir, permitindo alterar dinamicamente o centro da plataforma a partir de um clique na imagem. Embora não reenvie comandos físicos aos motores, este mecanismo visual melhora a flexibilidade nos testes;

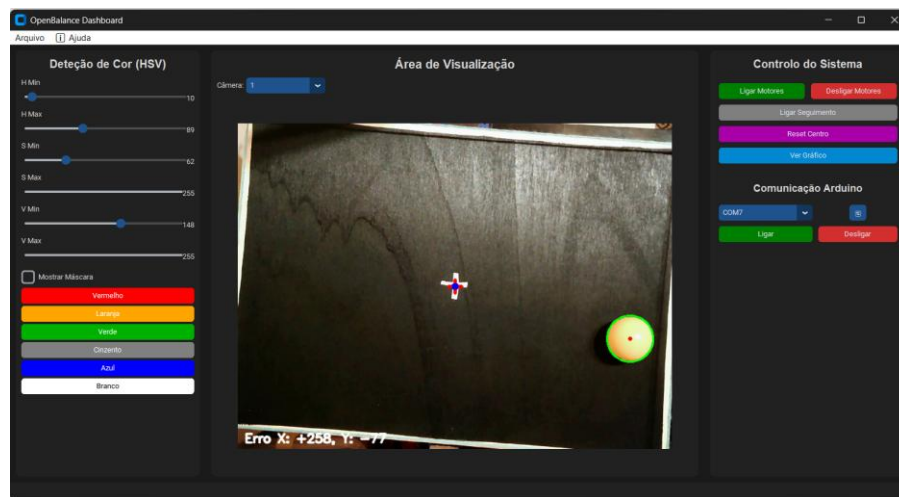


Figura 7 - Interface gráfica da dashboard OpenBalance (v1.0)

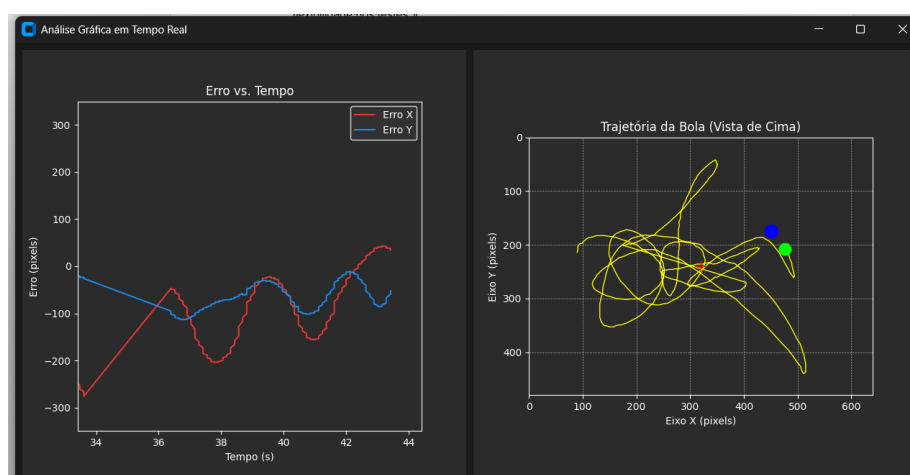


Figura 8 – Análise gráfica em tempo real

Além disso, a aplicação permite guardar e carregar configurações em ficheiros JSON (HSV), mudar entre câmaras disponíveis, e navegar por menus de ajuda, manual e créditos. A interface gráfica foi desenhada para ser robusta, pedagógica e extensível, proporcionando uma ferramenta visual eficaz para afinação, testes e demonstrações educativas.

Testes e resultados preliminares

Foram realizados diversos testes em condições reais. Os principais resultados incluem:

- **Deteção visual:** eficaz com bolas de cores contrastantes (ex: laranja ou amarela), alcançando taxas de 20–25 FPS com boa estabilidade sob luz difusa.
- **Estabilização:** com os parâmetros PID corretamente definidos (por ZN), a bola foi capaz de regressar ao centro após pequenas perturbações e manter-se em equilíbrio durante longos períodos.
- **Resposta física:** os servos DM996 responderam com precisão, e os limites físicos evitaram movimentos excessivos ou colisões estruturais.
- **Calibração visual assistida:** a interface permitiu observar em tempo real os efeitos das alterações nos parâmetros, facilitando a afinação por tentativa-erro.

A documentação do processo, incluindo valores experimentais, presets, vídeos e capturas de ecrã, foi organizada num repositório público no GitHub, promovendo a transparência e a replicabilidade do projeto.

Procedimento Proposto para Aplicação de Algoritmos de Otimização e Aprendizagem

Com vista à futura integração de algoritmos de otimização e aprendizagem automática no sistema OpenBalance, propõe-se o seguinte procedimento experimental, orientado para a aplicação direta no ambiente físico. Este processo permite afinar controladores, como o PID, ou treinar agentes de Reinforcement Learning, com base em dados reais obtidos durante a operação da plataforma.

1. Definição do Espaço de Parâmetros

Inicia-se por identificar os parâmetros a otimizar, como os ganhos K_p , K_i e K_d no caso de um controlo PID, ou a política de ações no caso de métodos de aprendizagem por reforço. Devem ser estabelecidos intervalos admissíveis para cada parâmetro, garantindo segurança e viabilidade física do sistema.

2. Preparação do Sistema para Testes Automatizados

O sistema deve ser estabilizado e protegido contra falhas graves (ex: inclusão de um fio periférico para impedir a queda da bola). A comunicação entre a interface Python e o microcontrolador (Arduino) deve permitir o envio dinâmico de parâmetros de controlo. A deteção da posição da bola deverá funcionar de forma contínua, usando visão computacional.

3. Estruturação de um Ciclo de Episódio Experimental

Cada episódio corresponde à execução de um teste com um conjunto de parâmetros. O ciclo proposto inclui:

- Envio dos parâmetros de controlo para o Arduino.
- Início da deteção da posição da bola e ativação do controlo.
- Execução durante um tempo fixo (ex.: 10 segundos) ou até ocorrer uma falha (ex.: toque nos limites).
- Cálculo de métricas de desempenho: erro médio (distância ao centro), número de colisões com os limites e tempo até falha.
- Armazenamento dos resultados para posterior análise.

4. Critério de Término de Episódio

O episódio deve terminar caso:

- A bola toque nos limites da plataforma (detetado por proximidade aos extremos da imagem).
- Seja atingido um tempo máximo de execução (timeout).

Estas condições asseguram a integridade física da plataforma e evitam repetições desnecessárias em episódios inviáveis.

5. Aplicação do Algoritmo de Otimização

De acordo com a abordagem escolhida, o sistema poderá:

- Utilizar **Hill Climbing** para testar variações incrementais dos parâmetros e aceitar apenas as que melhoram o desempenho.
- Empregar **Q-Learning** para construir uma política discreta com base em estados e recompensas reais.
- Recorrer a algoritmos avançados como **DDPG** ou **PPO** para aprender políticas contínuas através de redes neuronais.

Todas as decisões são tomadas com base em dados reais de desempenho do sistema físico.

6. Visualização e Registo de Resultados

Os dados de cada episódio (parâmetros testados, erro obtido, tempo até falha, número de colisões) devem ser registados automaticamente. Gráficos de evolução do erro e comparação de diferentes estratégias serão úteis para análise e validação dos algoritmos.

7. Validação Final e Avaliação de Robustez

Após a identificação de uma solução considerada ótima, esta deve ser testada em múltiplos episódios consecutivos, para avaliar a sua consistência, estabilidade e capacidade de adaptação a pequenas variações no sistema.

Este procedimento visa não só otimizar o desempenho da plataforma OpenBalance, como também criar uma base experimental sólida para a aplicação pedagógica de algoritmos de controlo inteligente em ambiente real. A sua implementação permitirá a exploração de tópicos avançados em robótica, controlo e inteligência artificial de forma integrada e aplicada.

5 CONCLUSÃO

O desenvolvimento do projeto OpenBalance permitiu integrar, de forma prática e aplicada, os conhecimentos adquiridos ao longo da unidade curricular de Laboratório de Aplicações em Robótica e Aprendizagem (PRIA). Através da construção de uma plataforma física capaz de equilibrar uma bola com base em visão computacional e controlo PID, foi possível explorar conceitos fundamentais de automação, eletrónica, programação e inteligência artificial aplicada.

A abordagem seguida privilegiou a simplicidade, modularidade e reprodutibilidade, recorrendo a componentes acessíveis e software open source. A integração de uma interface gráfica interativa permitiu não apenas visualizar os dados em tempo real, mas também facilitar o processo de afinação e tornar o sistema mais compreensível e atrativo para fins didáticos. A utilização de Python com OpenCV e CustomTkinter, aliada à comunicação com o Arduino, demonstrou ser uma solução eficaz e versátil para projetos robóticos educativos.

Do ponto de vista pedagógico, o projeto destacou-se como uma ferramenta com forte aplicabilidade no ensino de controlo automático, visão computacional e robótica, sendo facilmente replicável em ambientes escolares e universitários. A possibilidade de observar em tempo real a resposta de um sistema PID a estímulos externos reforça a ligação entre teoria e prática, promovendo uma aprendizagem ativa, exploratória e motivadora.

Perspetivas Futuras

Com base na experiência adquirida, identificam-se várias direções possíveis para a evolução do projeto:

- Expansão da plataforma para sistemas com mais graus de liberdade;
- Adição de braços articulados com capacidade de interação ativa com a bola, como empurrar ou direcionar para zonas-alvo, abrindo portas a comportamentos com objetivos dinâmicos;
- Integração de algoritmos de aprendizagem por reforço, permitindo que o sistema aprenda estratégias ótimas de controlo com base na experiência e melhore progressivamente o desempenho;
- Substituição da visão computacional por uma superfície sensorizada (tátil ou resistiva), reduzindo a dependência da imagem e melhorando a latência da deteção;
- Utilização de câmaras de maior resolução e processamento embarcado com Raspberry Pi, tornando o sistema mais autónomo e portátil.

Do ponto de vista pessoal e académico, este projeto reforçou a importância da experimentação iterativa, da integração entre hardware e software, e do potencial das ferramentas open source para fins educativos. No futuro, gostaria de continuar a explorar formas de tornar sistemas como o OpenBalance mais autónomos, interativos e adaptativos, incorporando técnicas de aprendizagem automática e novas formas de interação sensorial.

BIBLIOGRAFIA

Controlo e Automação

- Ogata, K. (2010). *Modern Control Engineering* (5th ed.). Prentice Hall.
- Dorf, R. C., & Bishop, R. H. (2017). *Modern Control Systems* (13th ed.). Pearson.
- Nise, N. S. (2020). *Control Systems Engineering* (8th ed.). Wiley.

Robótica Educacional e Arduino

- Blum, J. (2013). *Exploring Arduino: Tools and Techniques for Engineering Wizardry*. Wiley.
- Monk, S. (2016). *Programming Arduino: Getting Started with Sketches* (2nd ed.). McGraw-Hill Education.

Python, OpenCV e Interfaces Gráficas

- Beyeler, M. (2017). *Machine Learning for OpenCV*. Packt Publishing.
- Laganière, R. (2014). *OpenCV 3 Computer Vision Application Programming Cookbook*. Packt Publishing.
- Chen, J. (2023). *Learn OpenCV with Python by Examples*.
- Harrison, M. (2017). *Tkinter GUI Programming by Example*. Packt Publishing.

Webgrafia

GitHub — JohanLink:

- <https://github.com/JohanLink/Ball-Balancing-PID-System>

GitHub — giusenso:

- <https://github.com/giusenso/Ball-Balancing-PID-System>

Thingiverse – Ball Balancing PID Robot:

- <https://www.thingiverse.com/thing:4457405>

Documentação oficial do OpenCV:

- <https://docs.opencv.org>

Biblioteca Adafruit PWM Servo Driver (PCA9685):

- <https://learn.adafruit.com/16-channel-pwm-servo-driver>

CustomTkinter – GUI Moderna para Python:

- <https://github.com/TomSchimansky/CustomTkinter>

Introdução ao Controlo PID (Control Engineering):

- <https://www.controleng.com/articles/pid-control-explained>

Curso de Controlo PID com Arduino (Maker Guides):

- <https://www.makerguides.com/arduino-pid-controller>

Prompts utilizados (ChatGPT / OpenAI)

Durante o processo de desenvolvimento e documentação, foram utilizados vários prompts para obter apoio técnico, estrutural e criativo, incluindo:

- “Cria uma estrutura de relatório técnico para um projeto de controlo PID com visão computacional.”
- “Explica como calibrar servos com Arduino usando PCA9685.”
- “Como implementar controlo PID em Python com comunicação serial para Arduino?”
- “Sugere uma interface gráfica com CustomTkinter para controlo PID e HSV em tempo real.”
- “Descreve os passos de testes e validação de um sistema que equilibra uma bola com feedback visual.”
- “Adiciona perspetivas futuras para uma plataforma de controlo com múltiplos braços e IA.”
- “Faz uma reflexão final para relatório técnico com foco pedagógico e open-source.”